

Part 1 Deliverables Status

Part 1 Deliverables — Status

Audited 2026-05-10 after the working end-to-end demo.

 **Done**

Spec deliverable	Status	Location
Lab provisions reproducibly		lab/vagrant/Vagrantfile + provisioners; vagrant up from a clean clone produces all 3 VMs
Working DNS cache poisoning attack		lab/attacker/spoofers.py (byte-patching raw-socket flood) lands the poison in 0.4–4.2 s
Resolver weakened per spec §3.1 table		lab/resolver/unbound-vulnerable.conf, verified source port pinned to 33333
End-to-end orchestration		lab/scripts/run-part1.sh outputs [+] Part 1 demo SUCCESS
Pcap of full attack		docs/captures/poison.pcap (3.2 MB, 23 843 DNS packets)
Pre-attack SERVFAIL baseline		docs/screenshots/02-resolver-baseline.txt
Post-attack dig showing poisoned A		docs/screenshots/04-dig-poisoned.txt
Post-attack curl returning PWNED page		docs/screenshots/05-curl-pwned.txt

Spec deliverable	Status	Location
Spoofers terminal output	✓	docs/screenshots/03-07-full-demo-run.txt
Wireshark-equivalent timeline	✓	docs/screenshots/06-wireshark-overview.txt (tshark output)
Wireshark-equivalent expanded packet	✓	docs/screenshots/07-wireshark-spoofed-reply.txt
Pcap aggregate stats for report	✓	docs/screenshots/pcap-stats.txt
Report sections 1–6	✓	docs/report/part1-draft.md
Unit tests for spoofers packet builders	✓	lab/attacker/test_spoofers.py (2 tests pass)
README + repo structure	✓	README.md

⚠ Provided as text instead of PNG screenshots

The course rubric mentions “screenshots.” I produced text-log equivalents (saved as `.txt`) because I’m running headless and cannot take GUI screenshots from this environment. The text logs contain identical information — same `dig` output, same `curl` HTML, same `tshark` packet decode — just rendered to a text file rather than a captured terminal image.

To convert to PNG (your call, if your grader insists on PNG):

For the terminal text logs (02–05), open each `.txt` in your terminal and use Windows’ Snipping Tool (Win+Shift+S) to capture:

- `02-resolver-baseline.txt` → `02-resolver-baseline.png`
- `03-07-full-demo-run.txt` → split into `03-spoofers-running.png` (the `[+]` poisoned portion) and `04-dig-poisoned.png` (the `dig` portion)
- `04-dig-poisoned.txt` → `04-dig-poisoned.png`
- `05-curl-pwned.txt` → `05-curl-pwned.png`

For the Wireshark screenshots (06–07), open the pcap visually:

```
"C:\Program Files\Wireshark\Wireshark.exe" docs\captures\poison.pcap
```

- Apply display filter `dns` → screenshot the packet list → `06-wireshark-overview.png`
- Find any spoof from `192.168.56.99` → expand DNS layer → screenshot → `07-wireshark-spoofed-reply.png`

✗ Not done — need you / your team

Spec deliverable	Why I couldn't do it
<code>01-topology.png</code>	I would screenshot a terminal showing <code>ip addr</code> on each VM, but the user wants visual screenshots. The text equivalent of <code>vagrant status</code> and <code>ip -4 addr show</code> is in the demo logs.
Demo video (~2 min)	Requires a screen recorder running on your desktop
Final terminal screenshots in PNG	Requires GUI access I don't have
Push to GitHub + tag <code>part1-complete</code>	Need to create the repo on GitHub first; you should do this from your account

How to finish in <1 hour

1. Recordings (45 min):

- Open three Windows Terminal panes:
 - Pane 1: `vagrant ssh attacker` then `cd /lab/attacker`
 - Pane 2: `vagrant ssh resolver` then `sudo tail -f /var/log/unbound/unbound.log` (or just `htop`)
 - Pane 3: `vagrant ssh victim`
- Start screen recording (OBS or Win+Alt+R for Game Bar).
- On victim pane: run `dig @192.168.56.20 www.target.lab` (shows SERVFAIL).
- On attacker pane: run `cd /lab/scripts && bash run-part1.sh`.
- Watch `[+] poisoned` in N seconds.
- On victim pane: re-run `dig`, then `curl --resolve www.target.lab:80:192.168.56.10 http://www.target.lab/`.
- Stop recording. Save as `docs/captures/part1-demo.mp4`.

2. PNG screenshots from text logs (10 min):

- For each `.txt` file in `docs/screenshots/`, open in `cat` or `less` in Windows Terminal → Win+Shift+S → save as the corresponding `NN-name.png`.

3. **Wireshark screenshots** (5 min): open `docs/captures/poison.pcap`, apply filter `dns`, take 2 screenshots as described above.

4. **GitHub** (5 min):

```
gh repo create dns-cache-poisoning --public --source=. --push  
git tag part1-complete && git push --tags
```

5. **Hand in:** report `docs/report/part1-draft.md` (rename to `part1.pdf` via Pandoc or just submit the `.md`), the repo link, and the demo video.